Experiment-5

Name: -P.Divyashree **Regno: -**192472291

Course Code: -MLA0305 **Slot-**B

Title: -Dynamic Programming Using Policy Iteration and Value Iteration

Aim-To implement Policy Iteration and Value Iteration using Dynamic Programming and determine the optimal policy and value function for a Reinforcement Learning environment.

Procedure

A. Policy Iteration

1. Define the environment and initialize a random policy.
2. Initialize the value function.
3. Perform policy evaluation using the Bellman equation.
4. Calculate the value of each state under the current policy.
5. Perform policy improvement.
6. Select the action with the maximum expected value.
7. Update the policy.
8. Repeat policy evaluation and policy improvement until the policy becomes stable.

B. Value Iteration

9. Initialize the value function.

$$V(s)=\max_{s'}\sum P(s'|s,a)[R(s,a,s')+\gamma V(s')]$$

11. Update the value function repeatedly.
12. Stop when the values converge.
13. Extract the optimal policy from the final value function.
14. Compare the results of Policy Iteration and Value Iteration.

Soursce Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture
from IPython.display import display, FileLink
from google.colab import files
np.random.seed(42)
print("=" * 80)
print("EXPERIMENT 5")
print("DYNAMIC PROGRAMMING USING POLICY ITERATION AND VALUE ITERATION")
print("=" * 80)
N = 500
base = np.column_stack([
    np.random.uniform(0, 15, N),
    np.random.uniform(0, 15, N),
```

```
    np.random.uniform(-5, 10, N),
    np.random.uniform(0.85, 0.99, N)
])

gmm = GaussianMixture(
    n_components=3,
    random_state=42
)

gmm.fit(base)

generated = gmm.sample(N)[0]

dataset = pd.DataFrame(
    generated,
    columns=[
        "State",
        "Next_State",
        "Reward",
        "Gamma"
    ]
)

dataset["State"] = dataset["State"].clip(0, 15).round().astype(int)
dataset["Next_State"] = dataset["Next_State"].clip(0,
15).round().astype(int)
dataset["Reward"] = dataset["Reward"].clip(-5, 10).round(2)
dataset["Gamma"] = dataset["Gamma"].clip(0.85, 0.99).round(3)

# ================================================================
# 2. 10 SAMPLE RECORDS
# ================================================================

print("\n10 SAMPLE DATASET RECORDS")

display(dataset.head(10))

print("Total Records:", len(dataset))

# ================================================================
# 3. DATASET SUMMARY
# ================================================================

display(
    dataset.describe().T[
```

```
        ["min", "max", "mean", "std"]
    ].round(3)
)

# ============================================================
# 4. DOWNLOAD
# ============================================================

filename = "Experiment_5_Dynamic_Programming_GMM_Dataset.csv"

dataset.to_csv(
    filename,
    index=False
)

print("\nDOWNLOAD DATASET:")

display(FileLink(filename))

files.download(filename)

# ============================================================
# 5. GRIDWORLD
# ============================================================

ROWS = 4
COLS = 4

states = [
    (r, c)
    for r in range(ROWS)
    for c in range(COLS)
]

GOAL = (3, 3)

OBSTACLES = [
    (1, 1),
    (2, 2)
]

ACTIONS = {
    "UP": (-1, 0),
    "DOWN": (1, 0),
    "LEFT": (0, -1),
```

```
    "RIGHT": (0, 1)
}

ARROWS = {
    "UP": "↑",
    "DOWN": "↓",
    "LEFT": "←",
    "RIGHT": "→"
}

GAMMA = 0.90

# ================================================================
# 6. TRANSITION
# ================================================================

def transition(state, action):

    if state == GOAL:
        return state, 0

    dr, dc = ACTIONS[action]

    nr = state[0] + dr
    nc = state[1] + dc

    if nr < 0 or nr >= ROWS or nc < 0 or nc >= COLS:
        return state, -2

    ns = (nr, nc)

    if ns in OBSTACLES:
        return state, -5

    if ns == GOAL:
        return ns, 10

    return ns, -1

# ================================================================
# 7. VALUE ITERATION
# ================================================================

V_vi = {
    s: 0.0
```

```
    for s in states
}

for o in OBSTACLES:
    V_vi[o] = np.nan

V_vi[GOAL] = 10

vi_delta_history = []

for vi_iteration in range(1, 501):

    new_V = V_vi.copy()
    delta = 0

    for s in states:

        if s in OBSTACLES or s == GOAL:
            continue

        values = []

        for a in ACTIONS:

            ns, reward = transition(s, a)

            q = reward + GAMMA * V_vi[ns]

            values.append(q)

        new_V[s] = max(values)

        delta = max(
            delta,
            abs(new_V[s] - V_vi[s])
        )

    V_vi = new_V

    vi_delta_history.append(delta)

    if delta < 0.001:
        break

# ============================================================
```

```python
# 8. VALUE ITERATION POLICY
# ============================================================

policy_vi = {}

for s in states:

    if s in OBSTACLES:
        policy_vi[s] = "BLOCK"
        continue

    if s == GOAL:
        policy_vi[s] = "GOAL"
        continue

    action_values = {}

    for a in ACTIONS:

        ns, reward = transition(s, a)

        action_values[a] = (
            reward +
            GAMMA * V_vi[ns]
        )

    policy_vi[s] = max(
        action_values,
        key=action_values.get
    )

# ============================================================
# 9. POLICY ITERATION
# ============================================================

V_pi = {
    s: 0.0
    for s in states
}

for o in OBSTACLES:
    V_pi[o] = np.nan

V_pi[GOAL] = 10
```

```
policy_pi = {}

for s in states:

    if s in OBSTACLES or s == GOAL:
        policy_pi[s] = None
    else:
        policy_pi[s] = "RIGHT"

pi_iteration_history = []

for pi_iteration in range(1, 101):

    # ----------------------------------------------------------
    # Policy Evaluation
    # ----------------------------------------------------------

    for _ in range(100):

        new_V = V_pi.copy()

        for s in states:

            if s in OBSTACLES or s == GOAL:
                continue

            ns, reward = transition(
                s,
                policy_pi[s]
            )

            new_V[s] = (
                reward +
                GAMMA * V_pi[ns]
            )

        V_pi = new_V

    # ----------------------------------------------------------
    # Policy Improvement
    # ----------------------------------------------------------

    stable = True

    for s in states:
```

```
            if s in OBSTACLES or s == GOAL:
                continue

            old_action = policy_pi[s]

            action_values = {}

            for a in ACTIONS:

                ns, reward = transition(s, a)

                action_values[a] = (
                    reward +
                    GAMMA * V_pi[ns]
                )

            best_action = max(
                action_values,
                key=action_values.get
            )

            policy_pi[s] = best_action

            if old_action != best_action:
                stable = False

        pi_iteration_history.append(pi_iteration)

        if stable:
            break

# ============================================================
# 10. RESULT COMPARISON
# ============================================================

valid_states = [
    s
    for s in states
    if s not in OBSTACLES
    and s != GOAL
]

vi_values = np.array([
    V_vi[s]
```

```
    for s in valid_states
])

pi_values = np.array([
    V_pi[s]
    for s in valid_states
])

comparison = pd.DataFrame({
    "Algorithm": [
        "Value Iteration",
        "Policy Iteration"
    ],

    "Iterations": [
        vi_iteration,
        pi_iteration
    ],

    "Maximum Value": [
        vi_values.max(),
        pi_values.max()
    ],

    "Minimum Value": [
        vi_values.min(),
        pi_values.min()
    ],

    "Average Value": [
        vi_values.mean(),
        pi_values.mean()
    ]
}).round(3)

print("\n" + "=" * 80)
print("POLICY ITERATION VS VALUE ITERATION")
print("=" * 80)

display(comparison)

# ============================================================
# 11. SUMMARY TABLE
# ============================================================
```

```
summary = pd.DataFrame({
    "Metric": [
        "Dataset Records",
        "Grid Size",
        "Total States",
        "Obstacles",
        "Goal",
        "Gamma",
        "VI Iterations",
        "PI Iterations",
        "VI Average Value",
        "PI Average Value"
    ],

    "Result": [
        len(dataset),
        "4 x 4",
        len(states),
        len(OBSTACLES),
        str(GOAL),
        GAMMA,
        vi_iteration,
        pi_iteration,
        round(vi_values.mean(), 3),
        round(pi_values.mean(), 3)
    ]
})

print("\nSUMMARY RESULTS")

display(summary)

# ============================================================
# 12. VISUALIZATION 1
# DATASET REWARD DISTRIBUTION
# ============================================================

plt.figure(figsize=(10, 6))

plt.hist(
    dataset["Reward"],
    bins=25,
    color="#845ef7",
    edgecolor="black"
)
```

```
plt.title(
    "GMM Synthetic Dataset - Reward Distribution",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Reward")
plt.ylabel("Frequency")

plt.show()

# ============================================================
# 13. VISUALIZATION 2
# VI CONVERGENCE
# ============================================================

plt.figure(figsize=(11, 6))

plt.plot(
    vi_delta_history,
    color="#e03131",
    linewidth=3,
    marker="o",
    markersize=3
)

plt.title(
    "Value Iteration Convergence",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Iteration")
plt.ylabel("Maximum Change")

plt.grid(alpha=0.25)

plt.show()

# ============================================================
# 14. VISUALIZATION 3
# ITERATION COMPARISON
# ============================================================
```

```
plt.figure(figsize=(9, 6))

plt.bar(
    ["Value Iteration", "Policy Iteration"],
    [vi_iteration, pi_iteration],
    color=["#339af0", "#51cf66"],
    edgecolor="black"
)

plt.title(
    "Number of Iterations Comparison",
    fontsize=16,
    fontweight="bold"
)

plt.ylabel("Iterations")

plt.show()

# ============================================================
# 15. VISUALIZATION 4
# VALUE COMPARISON
# ============================================================

plt.figure(figsize=(11, 6))

x = np.arange(len(valid_states))

plt.plot(
    x,
    vi_values,
    marker="o",
    linewidth=3,
    color="#339af0",
    label="Value Iteration"
)

plt.plot(
    x,
    pi_values,
    marker="s",
    linewidth=3,
    color="#ff922b",
    label="Policy Iteration"
)
```

```
plt.title(
    "State Value Comparison",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("State Index")
plt.ylabel("State Value")

plt.legend()
plt.grid(alpha=0.25)

plt.show()

# ================================================================
# 16. VISUALIZATION 5
# POLICY MAPS
# ================================================================

fig, axes = plt.subplots(
    1,
    2,
    figsize=(14, 6)
)

for ax, policy, title in [
    (axes[0], policy_vi, "Value Iteration Policy"),
    (axes[1], policy_pi, "Policy Iteration Policy")
]:

    values = np.zeros(
        (ROWS, COLS)
    )

    for r in range(ROWS):
        for c in range(COLS):

            if (r, c) in OBSTACLES:
                values[r, c] = np.nan
            else:
                values[r, c] = 1

    ax.imshow(
        values,
```

```
            cmap="viridis"
        )

        for r in range(ROWS):
            for c in range(COLS):

                state = (r, c)

                if state in OBSTACLES:
                    text = "■"
                elif state == GOAL:
                    text = "★"
                else:
                    text = ARROWS[policy[state]]

                ax.text(
                    c,
                    r,
                    text,
                    ha="center",
                    va="center",
                    fontsize=25,
                    color="white",
                    fontweight="bold"
                )

        ax.set_title(
            title,
            fontsize=15,
            fontweight="bold"
        )

plt.tight_layout()
plt.show()

print("\nExperiment 5 completed successfully.")
```

	State	Next_State	Reward	Gamma
0	6	0	-0.89	0.938
1	5	9	-2.71	0.850
2	7	15	-0.81	0.935
3	2	12	-4.97	0.914
4	1	9	-0.90	0.929
5	9	11	-1.54	0.908
6	13	7	-2.69	0.873
7	0	5	-2.06	0.850
8	8	12	-4.05	0.925
9	5	8	0.57	0.895

FIG: - 1 SAMPLE DATA

	Algorithm	Iterations	Maximum Value	Minimum Value	Average Value
0	Value Iteration	7	19.0	7.124	13.202
1	Policy Iteration	5	19.0	7.124	13.202

FIG: - 2 ALGORITHM COMPARISON

SUMMARY RESULTS

	Metric	Result
0	Dataset Records	500
1	Grid Size	4 x 4
2	Total States	16
3	Obstacles	2
4	Goal	(3, 3)
5	Gamma	0.9
6	VI Iterations	7
7	PI Iterations	5
8	VI Average Value	13.202
9	PI Average Value	13.202

FIG: - 3 SUMMARY RESULTS

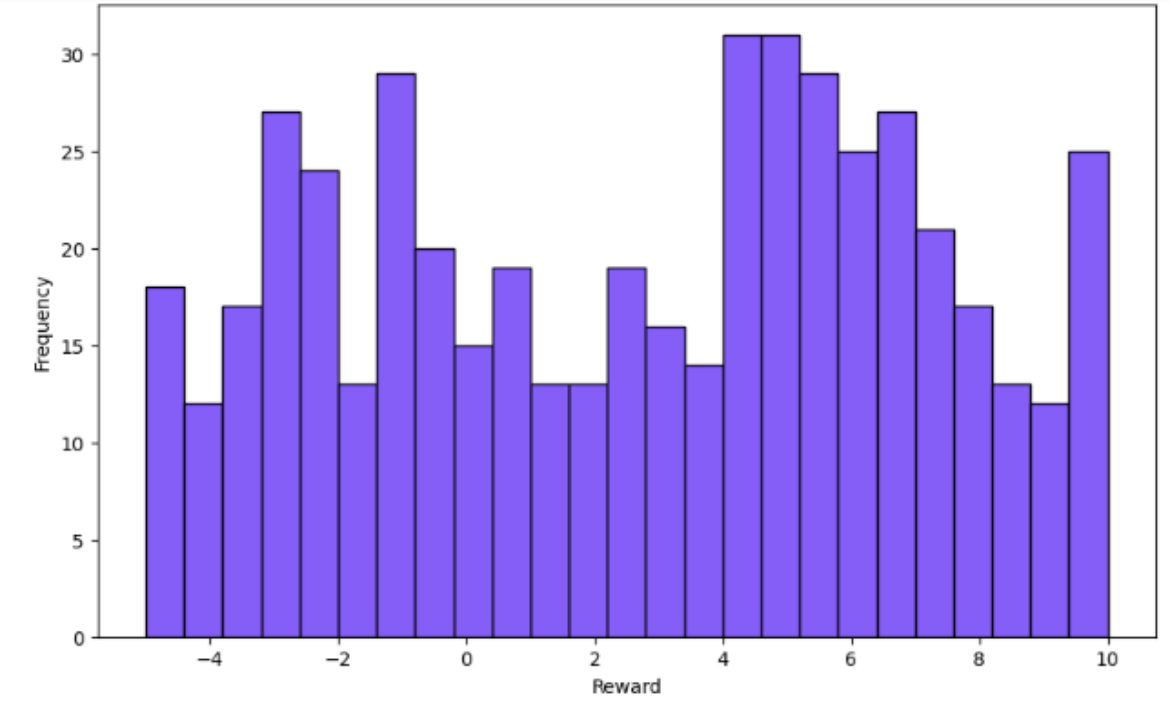

FIG: - 4 DATA SET REWARD DISTRIBUTION

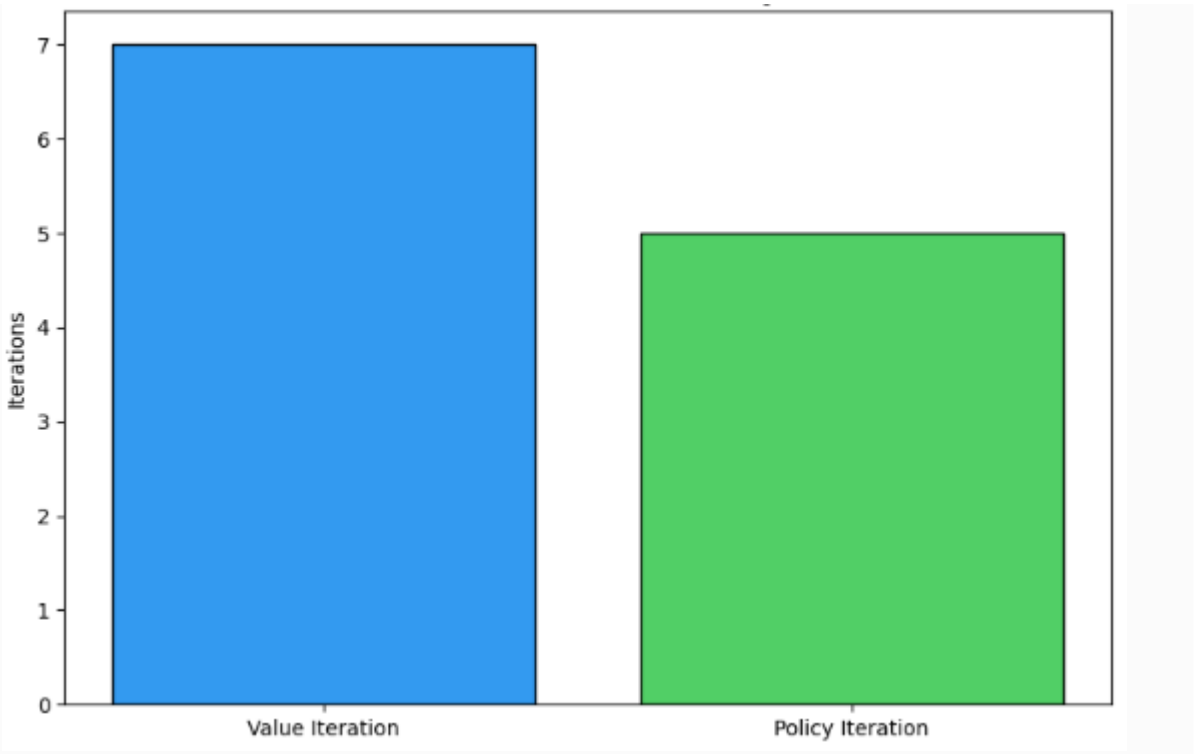

FIG: - 5 VISUALIZATION OF ALGORITHM COMPARISON

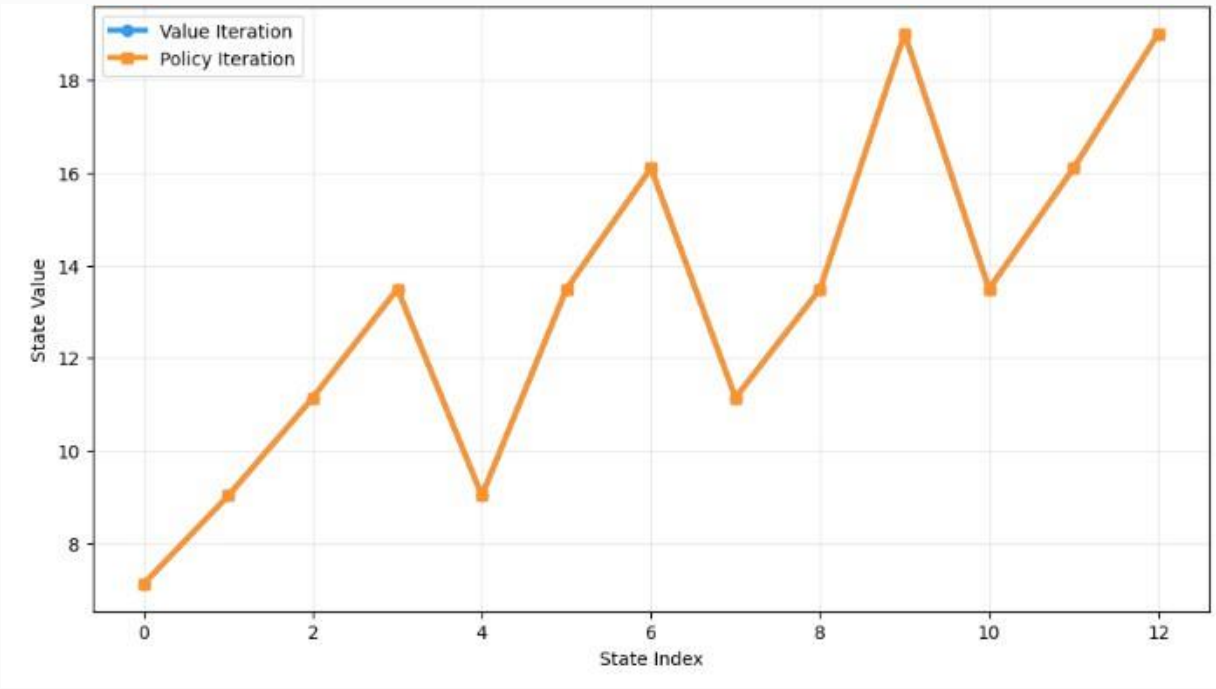

FIG: - 6 STATE VALUE COMPARISON

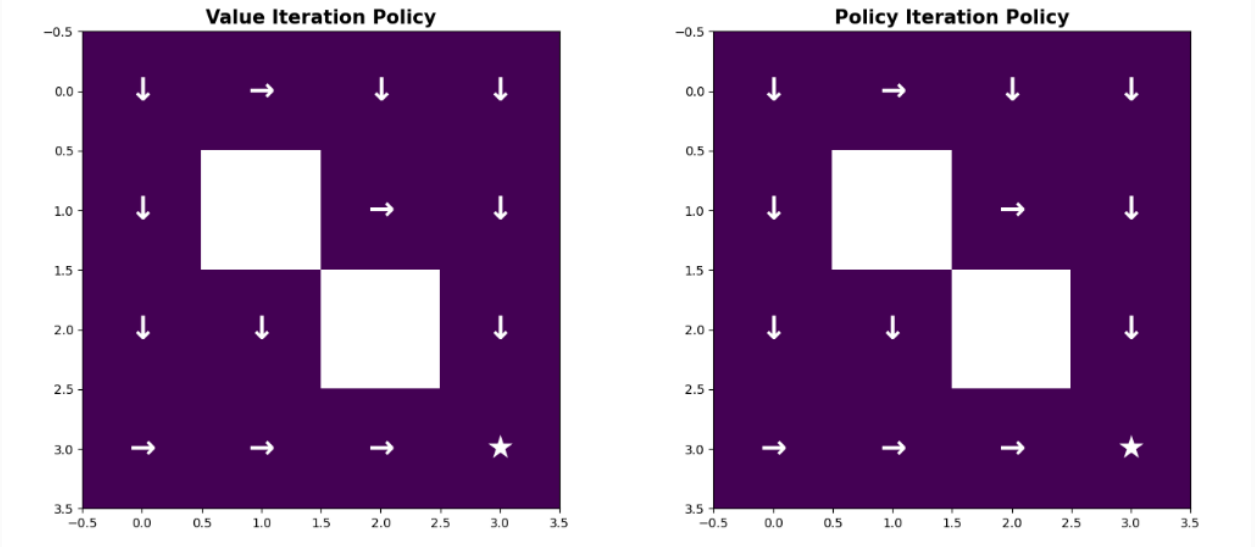

FIG: - 7 FINAL COMPARISON

Result

Both Policy Iteration and Value Iteration were successfully implemented. The optimal value function and optimal policy were obtained, and their performance was compared.